

AI-Powered Data Quality Assistant: 3-Day Case Study

Overview

Create an AI-powered data quality assistant that helps domain experts define and manage data quality rules for PostgreSQL databases using Great Expectations, without requiring deep technical knowledge of the framework.

Core Objectives

1. Demonstrate ability to build AI-first solutions
2. Show effective use of AI coding tools (e.g., Cursor, Copilot)
3. Showcase product thinking and user experience design
4. Create a functional MVP that solves a real problem

Technical Requirements

Backend Components

1. AI Rule Generator
 - Analyze table schema and sample data automatically
 - Generate recommended Great Expectations rules based on data patterns
 - Convert natural language descriptions into Great Expectations configurations
 - Store and manage generated rules
2. Data Quality Engine
 - Integration with Great Expectations framework
 - Execute rules against specified tables

- Generate comprehensive quality reports
- Cache results for quick viewing

Frontend Components

1. Table Explorer
 - List available database tables
 - Show basic table metadata and schema
 - Allow table selection for quality analysis
2. Rule Management Interface
 - Display AI-recommended rules for selected table
 - Accept natural language input for custom rules
 - Show generated Great Expectations configurations
 - Enable rule editing/customization
3. Results Dashboard
 - Display rule execution results
 - Highlight failures and warnings
 - Provide detailed error information
 - Enable quick re-running of rules

Proposed day by day evolution - (candidates aren't expected to follow this..this is just an outline)

Day 1: Foundation & Core AI Integration

- Set up project infrastructure
- Implement basic database connectivity
- Create AI prompt engineering for rule generation
- Build initial API endpoints
- Start frontend scaffolding

Day 2: Core Functionality

- Complete rule generation system
- Implement Great Expectations integration

- Build basic frontend components
- Create rule execution pipeline
- Implement results storage

Day 3: Polish & Enhancement

- Refine UI/UX
- Add automatic rule suggestions
- Implement caching and performance optimizations
- Add error handling and validation
- Create basic documentation

Evaluation Criteria - Equal Weightage

AI-First Development

- Effective use of AI coding tools
- Quality of AI integrations
- Prompt engineering approach
- AI-driven automation

Product Thinking

- User experience design
- Feature prioritization
- Error handling and edge cases
- Overall solution architecture

Technical Implementation

- Code quality and organization
- System performance
- API design
- Documentation

Technical Stack

Required

- Python (Backend)
- PostgreSQL (Use Supabase on Cloud)
- Great Expectations (Open Source)
- React/Next.js (Frontend)
- OpenAI API (Claude is also ok.) - Please ask for API key if you don't have a trial or its not sufficient

Recommended Tools

- Cursor
- GitHub Copilot
- Any other AI coding assistants

Deliverables

1. Working MVP application
1. Source code repository
1. Brief documentation explaining:
 - Architecture decisions
 - AI integration approach
 - Future enhancement possibilities
 - How AI tools were leveraged during development

Notes for Candidates

- Focus on leveraging AI tools for development
- Prioritize working features over perfect code
- Document your AI tool usage and decision-making
- Consider user experience for non-technical users
- Think about scalability and maintainability
- Don't reinvent the wheel - use existing libraries and tools

Bonus Points

- Creative use of AI beyond basic requirements

- Innovative approaches to user experience
- Additional automated features
- Performance optimizations
- Thoughtful error handling
- Clear documentation of AI tool usage

This case study is designed to evaluate not just coding ability, but also how effectively candidates can leverage AI tools and think about AI-first product development. The focus is on creating a practical solution that demonstrates understanding of modern development practices and AI integration.